Imagine you've been working with Scruffy and running your build manually all day. Each time you wanted to compile the CoffeeScript application, you just ran the compiler from the command line:

```
> coffee -c -o compiled app
```

The -o option to coffee lets you specify the output directory for the compiled files.

Running the application was fairly easy:

```
> node compiled/server.js
```

Running the tests was also fairly easy:

```
> node compiled/tests.js
```

You've only had to remember two things—easy when there's one application, but not so easy when there are lots of applications. Imagine you're asked to work on another program that you've never seen before. Here's the directory structure:

```
├── lib
│   ├── highball.coffee
│   ├── cocktail.coffee
│   ├── julep.coffee
├── app
│   ├── punch.coffee
│   ├── fizz.coffee
│   ├── flip.coffee
├── vendor
│   ├── mug.coffee
│   ├── beaker.coffee
│   ├── teacup.coffee
├── resources
│   ├── reference.csv
```

How do you compile *that* program? Perhaps there's some documentation somewhere that tells you how to build it; it might even be up to date, perhaps not. For an application that lives in a single file, the build just means compiling a single file:

```
> coffee –c single_file_application.coffee
```